算法设计报告

题目：$CSP-S$ 2021 $T1$ 廊桥分配

题目描述

当一架飞机抵达机场时，可以停靠在航站楼旁的廊桥，也可以停靠在位于机场边缘的远机位。乘客一般更期待停靠在廊桥，因为这样省去了坐摆渡车前往航站楼的周折。然而，因为廊桥的数量有限，所以这样的愿望不总是能实现。

机场分为国内区和国际区，国内航班飞机只能停靠在国内区，国际航班飞机只能停靠在国际区。一部分廊桥属于国内区，其余的廊桥属于国际区。

L 市新建了一座机场，一共有 n 个廊桥。该机场决定，廊桥的使用遵循“先到先得”的原则，即每架飞机抵达后，如果相应的区（国内/国际）还有空闲的廊桥，就停靠在廊桥，否则停靠在远机位（假设远机位的数量充足）。该机场只有一条跑道，因此不存在两架飞机同时抵达的情况。

现给定未来一段时间飞机的抵达、离开时刻，请你负责将 n 个廊桥分配给国内区和国际区，使停靠廊桥的飞机数量最多。

输入格式

输入的第一行，包含三个正整数 n, m_1, m_2，分别表示廊桥的个数、国内航班飞机的数量、国际航班飞机的数量。

接下来 m_1 行，是国内航班的信息，第 i 行包含两个正整数 $a_{1.i}, b_{1.i}$ 分别表示一架国内航班飞机的抵达、离开时刻。

接下来 m_2 行，是国际航班的信息，第行 i 包含两个正整数 $a_{2.i}, b_{2.i}$ 分别表示一架国际航班飞机的抵达、离开时刻。

每行的多个整数由空格分隔。

输出格式

输出一个正整数，表示能够停靠廊桥的飞机数量的最大值。

输入输出样例

输入#1

```
3 5 4
1 5
3 8
6 10
9 14
13 18
2 11
4 15
7 17
12 16
```

输出 #1

```
7
```

输入 #2

```
2 4 6
20 30
40 50
21 22
41 42
1 19
2 18
3 4
5 6
7 8
9 10
```

输出 #2

```
4
```

数据范围

对于 20% 的数据，$n \le 100$，$m_1 + m_2 \le 100$。
对于 40% 的数据，$n \le 5000$，$m_1 + m_2 \le 5000$。
对于 100% 的数据，$1 \le n \le 10^5$，$m_1, m_2 \ge 1$，$m_1 + m_2 \le 10^5$ 所有 $a_{1,i}, b_{1,i}, a_{2,i}, b_{2,i}$ 为数值不超过 10^8 的互不相同的正整数，且保证对于每个 $i \in [1, m_1]$，都有 $a_{1,i} < b_{1,i}$，以及对于每个 $i \in [1, m_2]$，都有 $a_{2,i} < b_{2,i}$。

说明/提示

【样例解释 #1】

廊桥分配方案		国内航班飞机					国际航班飞机				停靠廊桥的飞机数量
国内区	国际区	1, 5	3, 8	6, 10	9, 14	13, 18	2, 11	4, 15	7, 17	12, 16	
0 个	3 个	✗	✗	✗	✗	✗	✓	✓	✓	✓	4
1 个	2 个	✓	✗	✓	✗	✓	✓	✓	✗	✓	6
2 个	1 个	✓	✓	✓	✓	✓	✓	✗	✗	✓	7
3 个	0 个	✓	✓	✓	✓	✓	✗	✗	✗	✗	5

在图中，我们用抵达、离开时刻的数对来代表一架飞机，如 $(1, 5)$ 表示时刻 1 抵达、时刻 5 离开的飞机；用 $\surd$ 表示该飞机停靠在廊桥，用 $\times$ 表示该飞机停靠在远机位。

我们以表格中阴影部分的计算方式为例，说明该表的含义。在这一部分中，国际区有 2 个廊桥，4 架国际航班飞机依如下次序抵达：

1. 首先 $(2, 11)$ 在时刻 2 抵达，停靠在廊桥。
2. 然后 $(4, 15)$ 在时刻 4 抵达，停靠在另一个廊桥。
3. 接着 $(7, 17)$ 在时刻 7 抵达，这时前 2 架飞机都还没离开、都还占用着廊桥，而国际区只有 2 个廊桥，所以只能停靠远机位。
4. 最后 $(12, 16)$ 在时刻 12 抵达，这时 $(2, 11)$ 这架飞机已经离开，所以有 1 个空闲的廊桥，该飞机可以停靠在廊桥。

根据表格中的计算结果，当国内区分配 2 个廊桥、国际区分配 1 个廊桥时，停靠廊桥的飞机数量最多，一共 7 架。

【样例解释 #2】

当国内区分配 2 个廊桥、国际区分配 0 个廊桥时，停靠廊桥的飞机数量最多，一共 4 架，即所有的国内航班飞机都能停靠在廊桥。

需要注意的是，本题中廊桥的使用遵循“先到先得”的原则，如果国际区只有 1 个廊桥，那么将被飞机 $(1, 19)$ 占用，而不会被 $(3, 4)$、$(5, 6)$、$(7, 8)$、$(9, 10)$ 这 4 架飞机先后使用。

问题分析

不难发现，国内区与国际区是两个完全相同的子问题。

最终的答案为 $max(f(x) + g(n - x))$,$f(x), g(x)$ 分别为国内区、国际区分配 x 个廊桥时能停靠的最大飞机数。

先只考虑一个区，比如国内区。不妨给每个廊桥依次编号为1、2、3.....m1。

假设廊桥数量没有限制，因为飞机停靠遵循“先到先得”原则，不妨让到达的飞机停靠至编号最小且空闲的廊桥。

注意到每个廊桥在整个过程中能停靠的飞机与廊桥总数、其他廊桥的活动均无关，每架飞机停靠位置亦如此。

因此可以维护一个集合 A （初始为所有国内飞机），表示还未选中的飞机。每次处理时将其中最先到达的飞机停入廊桥中，并将其从 A 中删除。对于分配 i 个廊桥时，只需在分配 $i - 1$ 个廊桥的基础上加上新增的能停入廊桥 i 的飞机数。

模型建立

用 $pair < a, b >$ 来存储每驾飞机的信息 a, b 分别为到达、离开时间。

用集合 A, B 来存储国内区与国际区仍停在远机位的飞机的集合

每次处理，重复做从远机位集合中取出最先到达的飞机，停入廊桥，并从集合中删除。

最终的答案为 $max(f(x) + g(n - x))$,$f(x), g(x)$ 分别为国内区、国际区分配 x 个廊桥时能停靠的最大飞机数。

算法设计

C++代码如下：

```
#include <bits/stdc++.h>
#define MAXN 100010
using namespace std;

int n, an, bn, aa[MAXN], bb[MAXN];
set<pair<int, int> > a, b;//国内区与国际区的远机位集合
pair<int, int> tmp;
set<pair<int, int> >::iterator id;

int ma()
{
    int t = 0, now = 0;
    while (true)
    {
        id = a.lower_bound(make_pair(now, now));//选中当前远机位中到达时间最早
        if (id == a.end())
            break;//若集合内飞机处理完，结束循环
```

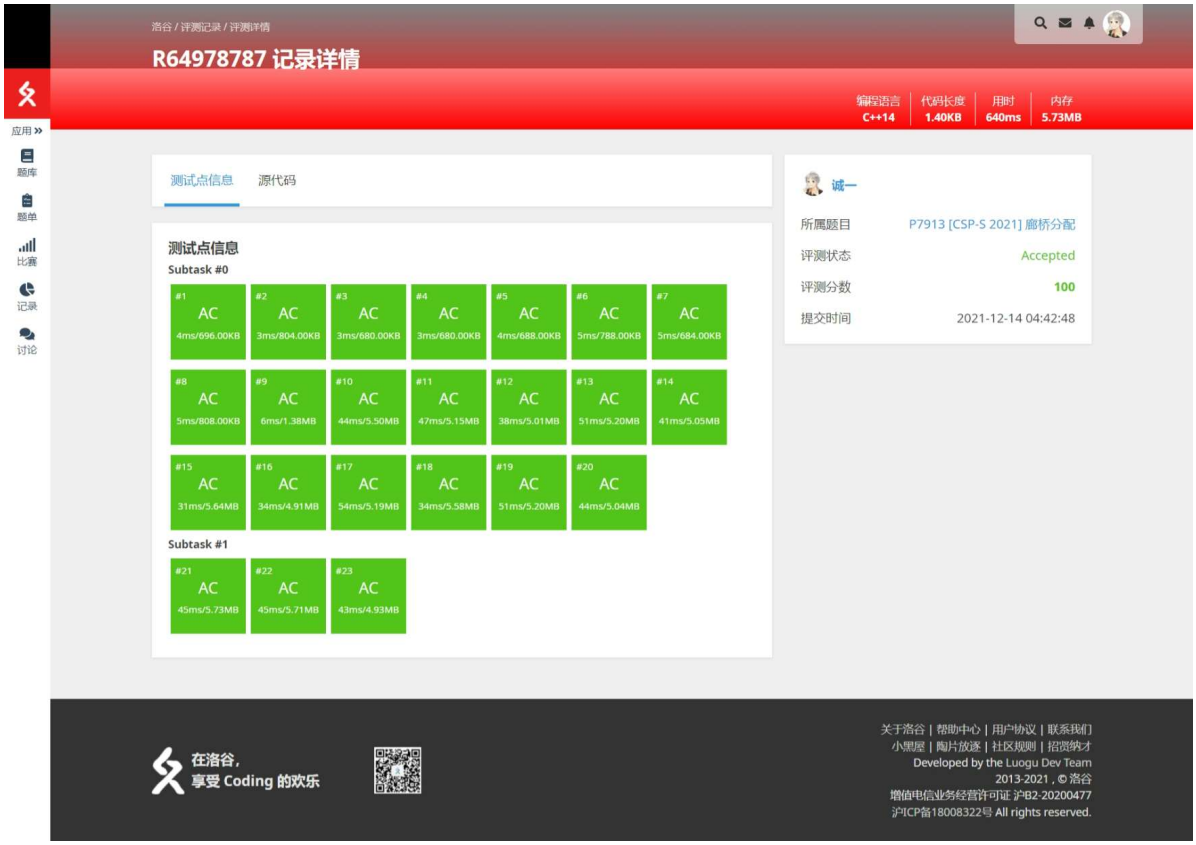
```
        t++;//统计新增停入廊桥的飞机数
        now = id->second;
        a.erase(id);//从远机位集合中删去刚才处理的飞机
    }
    return t;
}

int mb()
{
    int t = 0, now = 0;
    while (true)
    {
        id = b.lower_bound(make_pair(now, now));
        if (id == b.end())
            break;
        t++;
        now = id->second;
        b.erase(id);
    }
    return t;
}//与国内区同理

int main()
{
    scanf("%d%d%d", &n, &an, &bn);
    for (int i = 1; i <= an; i++)
    {
        scanf("%d%d", &tmp.first, &tmp.second);
        a.insert(tmp);
    }//读入国内区航班信息
    for (int i = 1; i <= bn; i++)
    {
        scanf("%d%d", &tmp.first, &tmp.second);
        b.insert(tmp);
    }//读入国际区航班信息
    aa[0] = 0;//边界值为0
    for (int i = 1; i <= n; i++)
        aa[i] = aa[i - 1] + ma();//处理国内区子问题
    bb[0] = 0;
    for (int i = 1; i <= n; i++)
        bb[i] = bb[i - 1] + mb();//处理国外区子问题
    int ans = 0;
    for (int i = 0; i <= n; i++)
        ans=max(ans, aa[i] + bb[n - i]);//计算最终答案，从边界0开始
    printf("%d\n", ans);
    return 0;
}
```

算法分析

时间复杂度为 $\Theta(n \log n)$


洛谷 / 评测记录 / 评测详情
R64978787 记录详情
编程语言 C++14
代码长度 1.40KB
用时 640ms
内存 5.73MB
应用
题库
题单
比赛
记录
讨论
测试点信息
源代码
测试点信息
Subtask #0
#1 AC 4ms/696.00KB
#2 AC 3ms/804.00KB
#3 AC 3ms/680.00KB
#4 AC 3ms/680.00KB
#5 AC 4ms/688.00KB
#6 AC 5ms/788.00KB
#7 AC 5ms/684.00KB
#8 AC 5ms/808.00KB
#9 AC 6ms/1.38MB
#10 AC 44ms/5.50MB
#11 AC 47ms/5.15MB
#12 AC 38ms/5.01MB
#13 AC 51ms/5.20MB
#14 AC 41ms/5.05MB
#15 AC 31ms/5.64MB
#16 AC 34ms/4.91MB
#17 AC 54ms/5.19MB
#18 AC 34ms/5.58MB
#19 AC 51ms/5.20MB
#20 AC 44ms/5.04MB
Subtask #1
#21 AC 45ms/5.73MB
#22 AC 45ms/5.71MB
#23 AC 43ms/4.93MB
诚一
所属题目 P7913 [CSP-S 2021] 廊桥分配
评测状态 Accepted
评测分数 100
提交时间 2021-12-14 04:42:48
在洛谷，
享受 Coding 的欢乐
关于洛谷 | 帮助中心 | 用户协议 | 联系我们
小黑屋 | 陶片放逐 | 社区规则 | 招贤纳才
Developed by the Luogu Dev Team
2013-2021 , © 洛谷
增值电信业务经营许可证 沪B2-20200477
沪ICP备18008322号 All rights reserved.